

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ANANYA BN (1BM24CS039)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum) Department of
Computer Science and Engineering



This is to certify that the Lab work entitled “ANALYSIS AND DESIGN OF ALGORITHMS” carried out by Ananya BN(1BM24CS039), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (23CS4PCADA) work prescribed for the said degree.

Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	10
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	13
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	16
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	22
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	26
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	29 31
9	Implement Fractional Knapsack using Greedy technique.	34
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	36
11	Implement "N-Queens Problem" using Backtracking.	38

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

void toposs(int V[MAX][MAX], int n) {
    int indegree[MAX];
    int check[MAX];
    int TP[MAX];
    int tp_count = 0;

    for (int i = 1; i <= n; i++) {
        indegree[i] = 0;
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (V[i][j] == 1) {
                indegree[j]++;
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        check[i] = 0;
```

```

}

while (1) {
    int w = -1;

    for (int i = 1; i <= n; i++) {
        if (check[i] == 0 && indegree[i] == 0) {
            w = i;
            break;
        }
    }

    if (w == -1) {
        break;
    }

    tp_count++;
    TP[tp_count] = w;

    check[w] = 1;

    for (int v = 1; v <= n; v++) {
        if (V[w][v] == 1) {
            indegree[v]--;
        }
    }
}

```

```

    }

    if (tp_count < n) {
        printf("NO Topological sequence\n");
    } else {

        printf("Topological Sequence: ");
        for (int i = 1; i <= tp_count; i++) {
            printf("%d ", TP[i]);
        }
        printf("\n");
    }
}

int main() {
    int v[100][100], n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &v[i][j]);
        }
    }
}

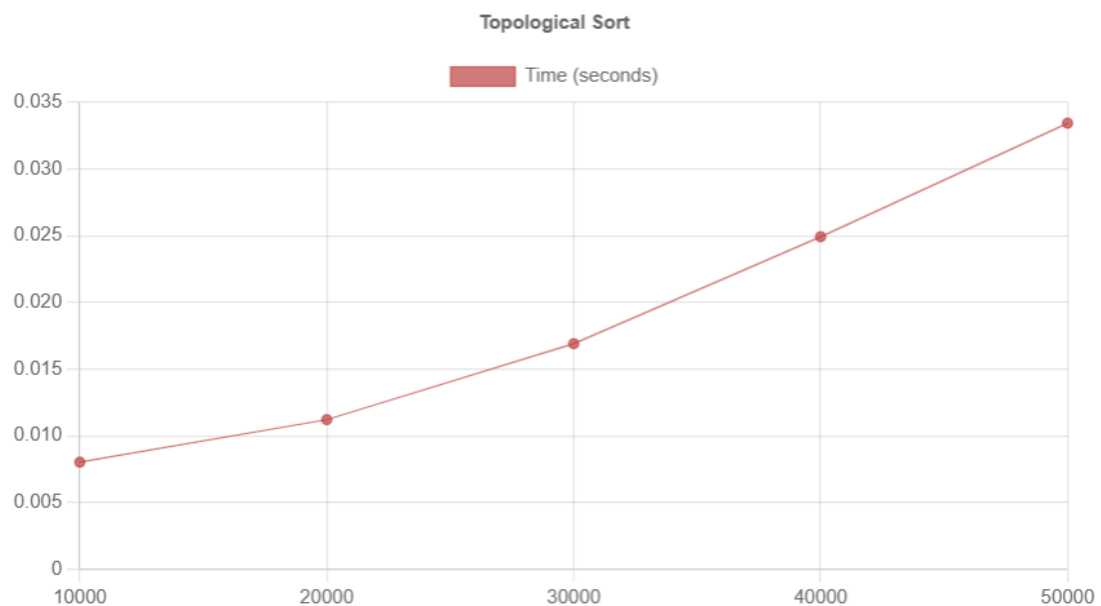
```

```
toposs(V, n);  
return 0;  
}
```

OUTPUT:

```
"C:\Users\Ananya B.N\C × + ▾ − □ ×  
Enter number of vertices: 4  
Enter number of edges (directed): 4  
Enter edges (format: source destination):  
0 1  
0 2  
1 3  
2 3  
Topological Sequence: 1 2 3 4  
  
Process returned 0 (0x0)    execution time : 3  
0.071 s  
Press any key to continue.  
|
```

GRAPH:



Leetcode Problem (207.Course Schedule):

```
#include <stdbool.h>
#include <stdlib.h>

bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize)
{
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int* size = (int*)calloc(numCourses, sizeof(int));

    for (int i = 0; i < prerequisitesSize; i++)
    {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        size[b]++;
    }

    for (int i = 0; i < numCourses; i++)
    {
        graph[i] = (int*)malloc(size[i] * sizeof(int));
        size[i] = 0;
    }

    for (int i = 0; i < prerequisitesSize; i++)
    {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][size[b]++] = a;
        indegree[a]++;
    }

    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;

    // Push nodes with indegree 0
    for (int i = 0; i < numCourses; i++)
    {
        if (indegree[i] == 0)
        {
            queue[rear++] = i;
        }
    }
}
```



```

int count = 0;

// Kahn's Algorithm (BFS)
while (front < rear)
{
    int node = queue[front++];
    count++;

    for (int i = 0; i < size[node]; i++)
    {
        int next = graph[node][i];
        indegree[next]--;

        if (indegree[next] == 0)
        {
            queue[rear++] = next;
        }
    }
}

for (int i = 0; i < numCourses; i++)
{
    free(graph[i]);
}

free(graph);
free(indegree);
free(size);
free(queue);

return count == numCourses;
}

```

OUTPUT:

Accepted Runtime: 0 ms

Case 1 Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Accepted Runtime: 0 ms

Case 1 Case 2

Input

numCourses =
2

prerequisites =
[[1,0],[0,1]]

Output

false

Expected

false

</> Code | Accepted ×

← All Submissions

Accepted 54 / 54 testcases passed

y3DGSPkiEw submitted at Mar 02, 2026 14:56

Analysis Solution

Runtime 37 ms | Beats 36.00%

Memory 110.42 MB | Beats 17.83%

Code | C

Testcase Test Result

You must run your code first

Lab program 2: Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX 20

#define LEFT 0
#define RIGHT 1

int val[MAX];
int dir[MAX];

void printPermutation(int n) {
    for (int i = 1; i <= n; i++) {
        printf("%d ", val[i]);
    }
    printf("\n");
}

void johnsonTrotter(int n) {

    for (int i = 1; i <= n; i++) {
        val[i] = i;
        dir[i] = LEFT;
    }

    printPermutation(n);

    while (true) {
        int mobile_val = -1;
        int mobile_idx = -1;
```

```

for (int i = 1; i <= n; i++) {
    // Case A: Arrow points LEFT
    if (dir[i] == LEFT && i > 1) {
        if (val[i] > val[i - 1] && val[i] > mobile_val) {
            mobile_val = val[i];
            mobile_idx = i;
        }
    }

    else if (dir[i] == RIGHT && i < n) {
        if (val[i] > val[i + 1] && val[i] > mobile_val) {
            mobile_val = val[i];
            mobile_idx = i;
        }
    }
}

if (mobile_idx == -1) {
    break;
}

int next_idx = (dir[mobile_idx] == LEFT) ? (mobile_idx - 1) : (mobile_idx + 1);

int temp_val = val[mobile_idx];
val[mobile_idx] = val[next_idx];
val[next_idx] = temp_val;

int temp_dir = dir[mobile_idx];
dir[mobile_idx] = dir[next_idx];

```

```

    dir[next_idx] = temp_dir;

    int current_k_idx = next_idx;

    for (int i = 1; i <= n; i++) {
        if (val[i] > mobile_val) {
            if (dir[i] == LEFT) {
                dir[i] = RIGHT;
            } else {
                dir[i] = LEFT;
            }
        }
    }

    printPermutation(n);
}

int main() {
    int n;

    printf("Enter number of elements (n): ");
    scanf("%d", &n);

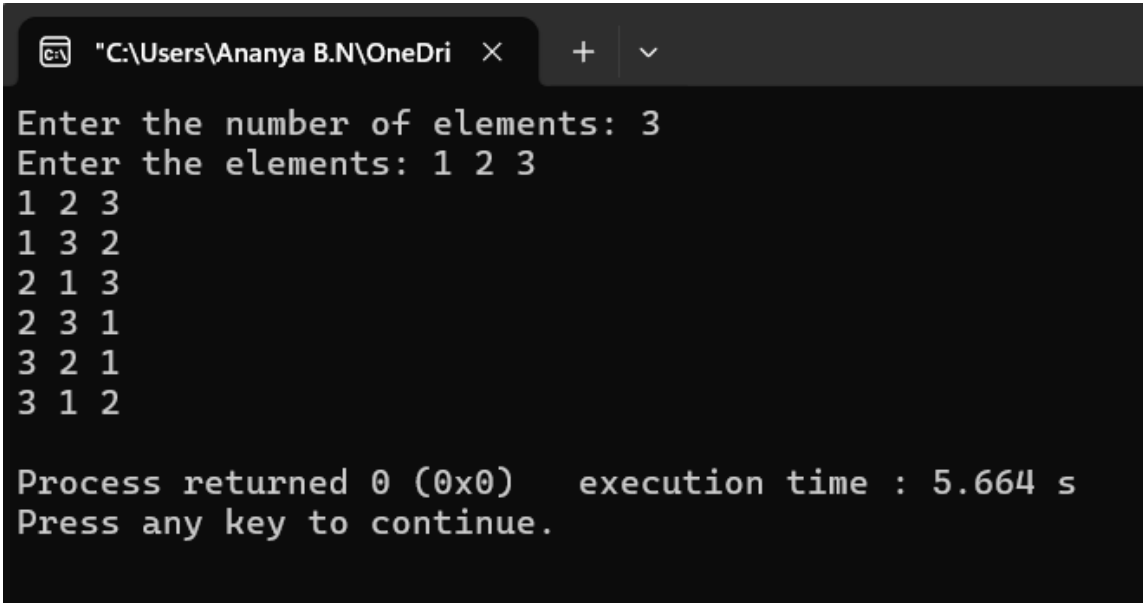
    if (n <= 0 || n >= MAX) {
        printf("Please enter a valid number of elements between 1 and %d.\n", MAX - 1);
        return 1;
    }

    printf("\nAll generated permutations using Johnson-Trotter:\n");
    johnsonTrotter(n);
}

```

```
    return 0;  
}
```

OUTPUT:

A screenshot of a C++ IDE window. The title bar shows the file path "C:\Users\Ananya B.N\OneDri" and standard window controls. The main area displays the program's output in a monospaced font. The output shows the user entering 3 for the number of elements and 1 2 3 for the elements. It then lists all six possible permutations of these three numbers. At the bottom, it shows the process returned 0 (0x0) with an execution time of 5.664 s and prompts the user to press any key to continue.

```
"C:\Users\Ananya B.N\OneDri" × + ▾  
Enter the number of elements: 3  
Enter the elements: 1 2 3  
1 2 3  
1 3 2  
2 1 3  
2 3 1  
3 2 1  
3 1 2  
  
Process returned 0 (0x0)   execution time : 5.664 s  
Press any key to continue.
```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>
```

```
void merge(int a[],int low,int high,int mid)
```

```
{
```

```
    int i,j,k,b[50];
```

```
    i=low;
```

```
    j=mid+1;
```

```
    k=low;
```

```
    while(i<=mid && j<=high)
```

```
    {
```

```
        if(a[i]<a[j])
```

```
        {
```

```
            b[k]=a[i];
```

```
            k++;
```

```
            i++;
```

```
        }
```

```
    else
```

```
    {
```

```
        b[k]=a[j];
```

```
        k++;
```

```
        j++;
```

```
    }
```

```
}
```

```
while(i<=mid)
```

```
{
```

```
    b[k]=a[i];
```

```

        k++;
        i++;
    }
    while(j<=high)
    {
        b[k]=a[j];
        k++;
        j++;
    }
    for(int x=low;x<=high;x++)
    {
        a[x]=b[x];
    }

}

void mergesort(int a[],int low,int high)
{
    int mid;
    if(low<high) //this has to be IF not WHILE
    {
        mid=(low+high)/2;
        mergesort(a,low,mid);
        mergesort(a,mid+1,high);
        merge(a,low,high,mid);
    }
}

int main()
{ int n,a[100];
  printf("enter the number of elements in the array");
  scanf("%d",&n);
  printf("enter the original array:");

```



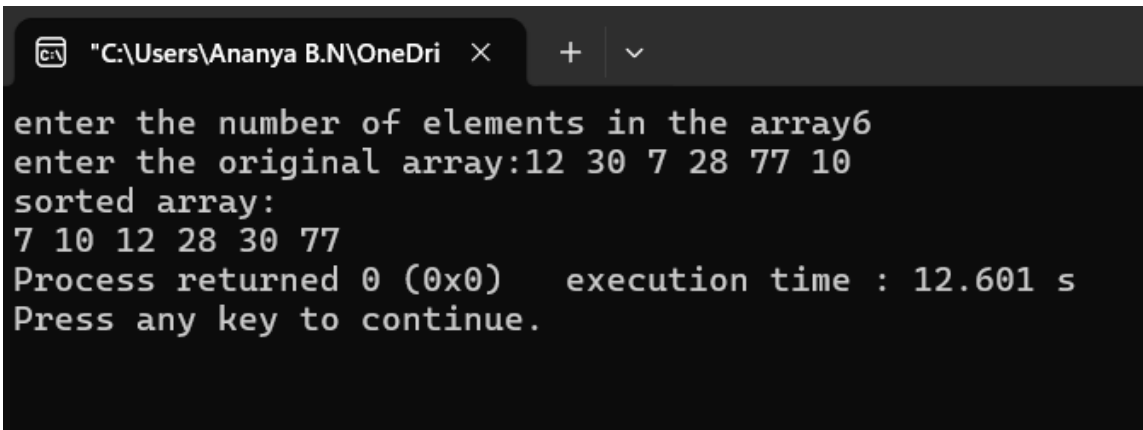
```

for(int i=0;i<n;i++)
{
    scanf("%d",&a[i]);
}
mergesort(a,0,n-1);
printf("sorted array:\n");
for(int i=0;i<n;i++)
{
    printf("%d ",a[i]);
}

return 0;
}

```

OUTPUT:

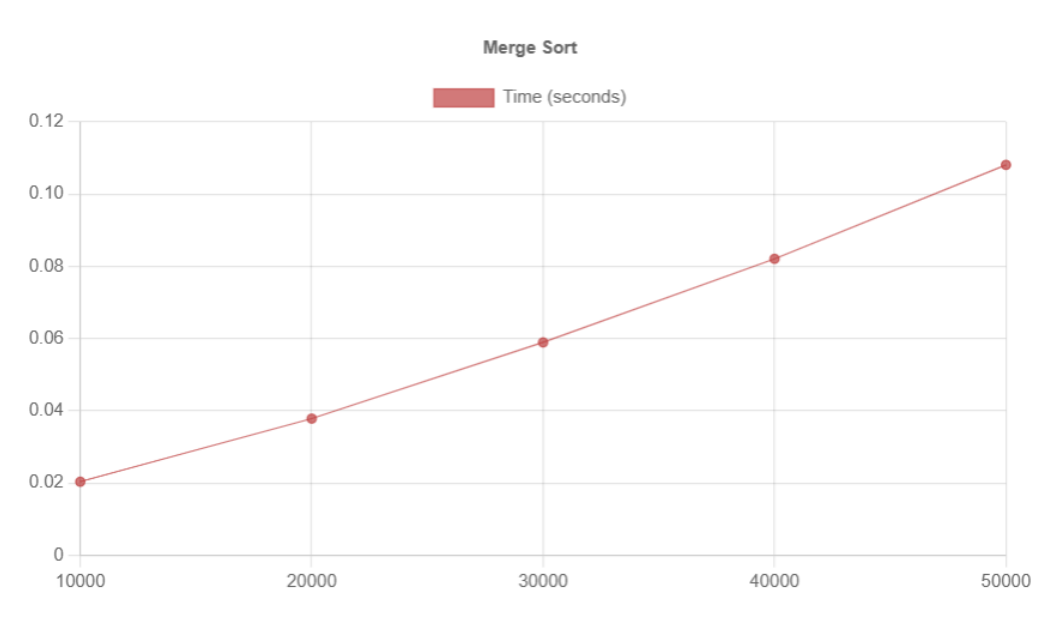


```

C:\Users\Ananya B.N\OneDri
enter the number of elements in the array6
enter the original array:12 30 7 28 77 10
sorted array:
7 10 12 28 30 77
Process returned 0 (0x0)    execution time : 12.601 s
Press any key to continue.

```

GRAPH:



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken

```
#include<stdio.h>

int partition(int a[],int low,int high)
{
    int i,j,key;
    key=a[low];
    i=low+1;
    j=high;
    while(1)
    {while(a[i]<=key && i<high)
    {
        i++;
    }
    while(a[j]>key)
    {
        j--;
    }
    if(i<j)
    {
        int temp;
        temp=a[i];
        a[i]=a[j];
        a[j]=temp;
    }
    else
    {
        int temp;
        temp=a[low];
        a[low]=a[j];
        a[j]=temp;
    }
}
```

```

        a[j]=temp;
        return j;
    }
}
}
void quicksort(int a[],int low,int high)
{
    int j;
    if(low<high)
    {j=partition(a,low,high);
    quicksort(a,low,j-1);
    quicksort(a,j+1,high);}
}
int main()
{
    int n,a[100];
    printf("enter the number of elements in the array");
    scanf("%d",&n);
    printf("enter the original array:");
    for(int i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    quicksort(a,0,n-1);
    printf("sorted array:\n");
    for(int i=0;i<n;i++)
    {
        printf("%d ",a[i]);
    }
    return 0;
}

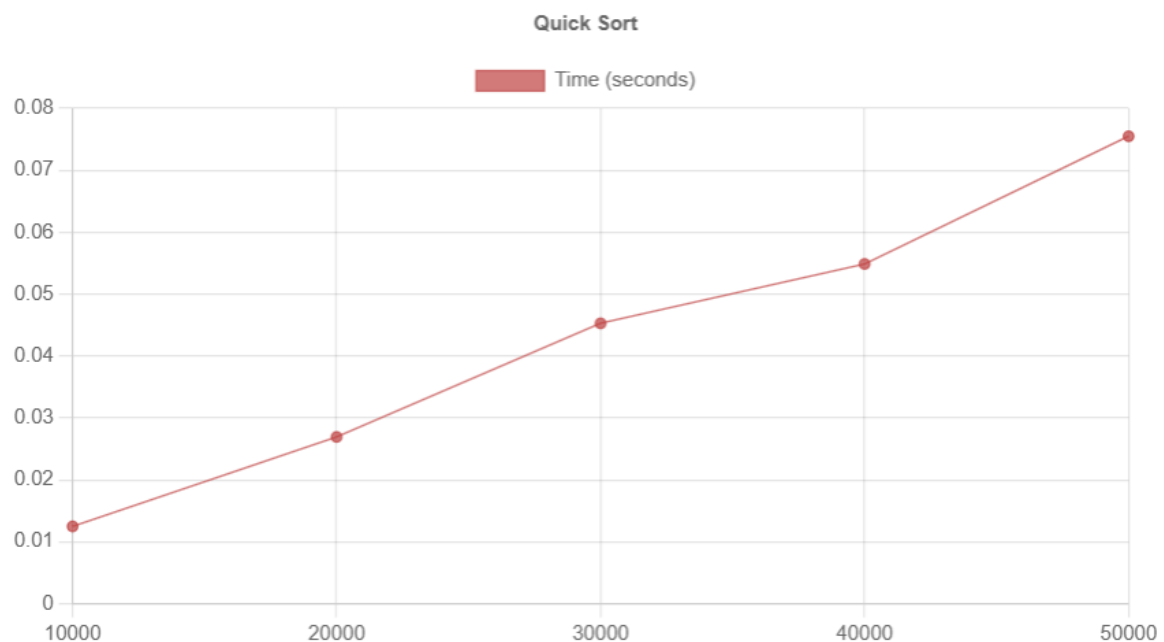
```

}

OUTPUT:

```
"C:\Users\Ananya B.N\OneDri  X + v
enter the number of elements in the array5
enter the original array:80 30 70 56 10
sorted array:
10 30 56 70 80
Process returned 0 (0x0)   execution time : 7.740 s
Press any key to continue.
|
```

GRAPH:



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>

void heapify(int a[],int n,int p)
{
    int c,item;
    item=a[p];
    c=2*p+1;
    while(c<=n-1)
    {
        if(c+1<=n-1)
        {
            if(a[c]<a[c+1])
            {
                c++;
            }
        }
        if(item<a[c])
        {
            a[p]=a[c];
            p=c;
            c=2*p+1;
        }
        else
        {
            break;
        }
    }
    a[p]=item;
```

```

    }
void build_heap(int a[],int n)
{
    int i;
    for(i=(n-1)/2;i>=0;i--)
    {
        heapify(a,n,i);
    }
}
void heapsort(int a[],int n)
{
    int i,temp;
    build_heap(a,n);
    for(i=n-1;i>0;i--)
    {
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,i,0);
    }

}
int main()
{
    int a[100],n,p;
    printf("enter the array size:");
    scanf("%d",&n);
    printf("enter the array");
    for(int i=0;i<n;i++){
        scanf("%d",&a[i]);}
    heapsort(a,n);

```

```

printf("after sort:");
for(int i=0;i<n;i++){
printf("%d ",a[i]);}

}

```

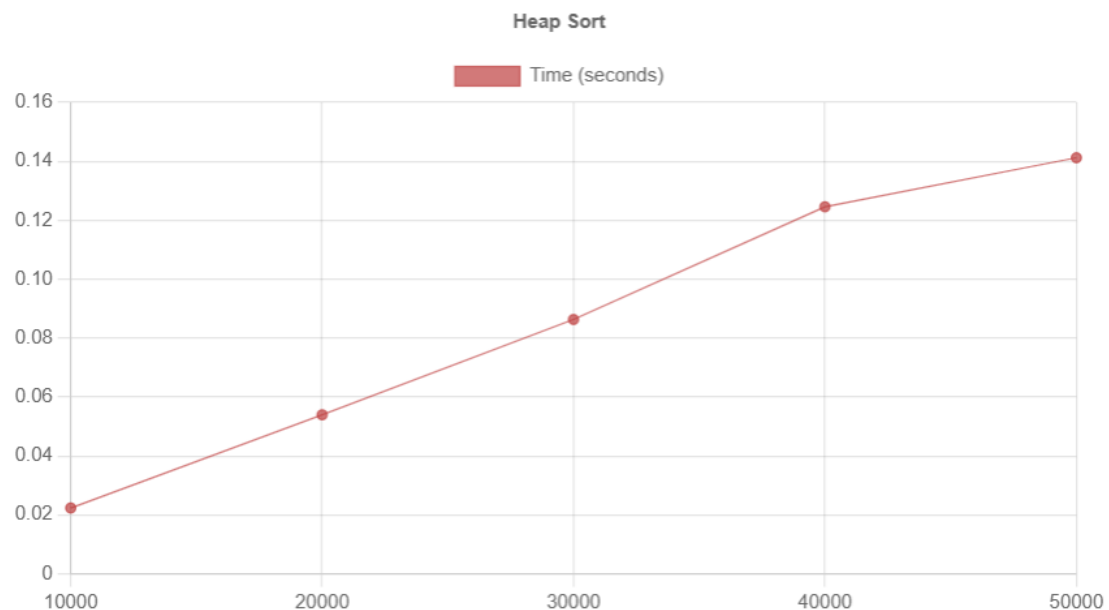
OUTPUT:

```

C:\Users\Ananya B.N\OneDri  ×  +  v
enter the array size:6
enter the array70 89 65 43 36 45
after sort:36 43 45 65 70 89
Process returned 0 (0x0)   execution time : 23.956 s
Press any key to continue.
|

```

GRAPH:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_ITEMS 100
#define MAX_WEIGHT 100

int max_value(int a, int b) {
    return (a > b) ? a : b;
}

int knapsackDp(int n, int w[MAX_ITEMS], int p[MAX_ITEMS], int M) {

    int Table[MAX_ITEMS][MAX_WEIGHT];
    for (int i = 0; i <= n; i++) {
        Table[i][0] = 0;    //first column as 0
    }

    for (int j = 0; j <= M; j++) {
        Table[0][j] = 0;    //first row as 0
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= M; j++) {

            if (j < w[i]) {
                Table[i][j] = Table[i - 1][j];
            }
        }
    }
}
```

```

        // Else take the maximum between excluding or including the current item
        else {
            Table[i][j] = max_value(Table[i - 1][j], Table[i - 1][j - w[i]] + p[i]);
        }

    }

}

return Table[n][M];
}

int main() {
    int n, M;
    int w[MAX_ITEMS];
    int p[MAX_ITEMS];

    printf("Enter number of items (n): ");
    scanf("%d", &n);

    printf("Enter maximum capacity of knapsack (M): ");
    scanf("%d", &M);

    printf("Enter weights of the items:\n");
    for (int i = 1; i <= n; i++) {
        printf("Weight for item %d: ", i);
        scanf("%d", &w[i]);
    }

    printf("Enter profits of the items:\n");
    for (int i = 1; i <= n; i++) {
        printf("Profit for item %d: ", i);
        scanf("%d", &p[i]);
    }
}

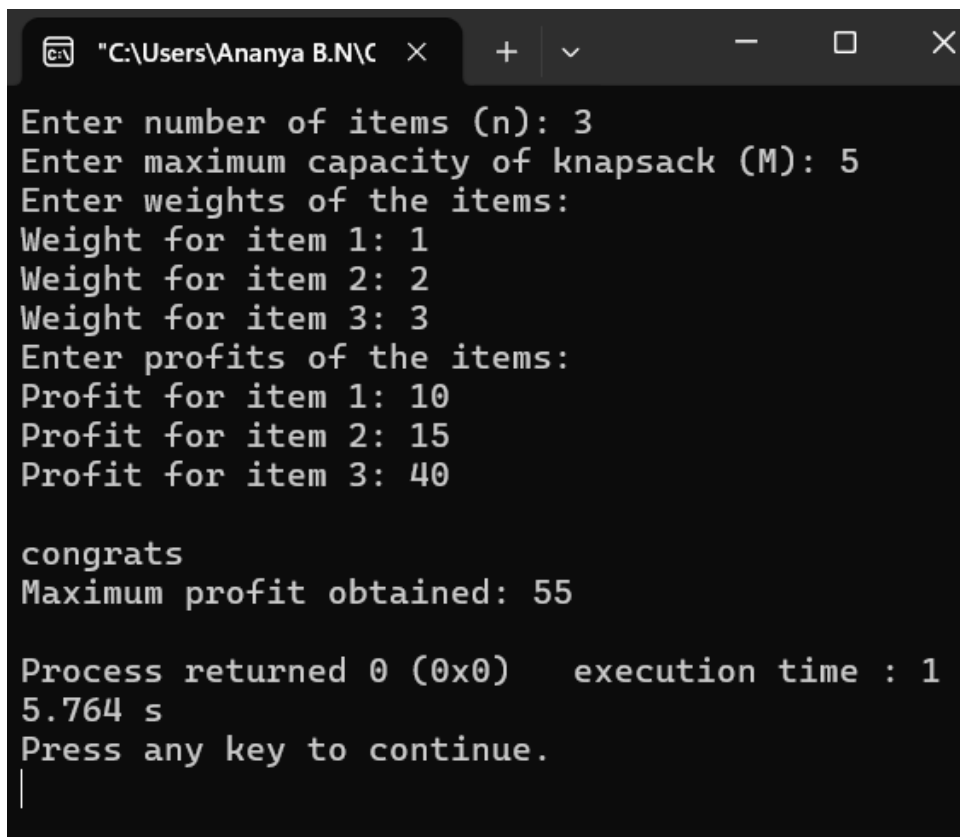
```

```
}

int max_profit = knapsackDp(n, w, p, M);

printf("\ncongrats\n");
printf("Maximum profit obtained: %d\n", max_profit);

return 0;
}
OUTPUT:
```



```
"C:\Users\Ananya B.N\C × + ▾ − □ ×
Enter number of items (n): 3
Enter maximum capacity of knapsack (M): 5
Enter weights of the items:
Weight for item 1: 1
Weight for item 2: 2
Weight for item 3: 3
Enter profits of the items:
Profit for item 1: 10
Profit for item 2: 15
Profit for item 3: 40

congrats
Maximum profit obtained: 55

Process returned 0 (0x0)    execution time : 1
5.764 s
Press any key to continue.
|
```

Leetcode Problem (801.Minimum Swaps):

```
#include <limits.h>
```

```
int min(int a, int b)
```

```
{  
    return (a < b) ? a : b;  
}
```

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size)
```

```
{  
    int keep = 0;  
    int swap = 1;  
    for (int i = 1; i < nums1Size; i++)  
    {  
        int newKeep = INT_MAX;  
        int newSwap = INT_MAX;  
  
        if (nums1[i] > nums1[i - 1] &&  
            nums2[i] > nums2[i - 1])  
        {  
            newKeep = min(newKeep, keep);  
            newSwap = min(newSwap, swap + 1);  
        }  
  
        if (nums1[i] > nums2[i - 1] &&  
            nums2[i] > nums1[i - 1])  
        {  
            newKeep = min(newKeep, swap);  
            newSwap = min(newSwap, keep + 1);  
        }  
    }  
}
```

```

    }
    keep = newKeep;
    swap = newSwap;
}
return min(keep, swap);
}

```

OUTPUT:

```
Accepted Runtime: 0 ms
Case 1 Case 2
Input
nums1 =
[1,3,5,4]
nums2 =
[1,2,3,7]
Output
1
Expected
1
```

Submission Details:

- Accepted: 117/117 testcases passed
- Submitted by: y3DGSPk1ew
- Submitted at: Jun 06, 2026 23:31
- Analysis: [Analysis](#)
- Solution: [Solution](#)

Performance Metrics:

- Runtime: 3 ms (Beats 48.68%)
- Memory: 16.50 MB (Beats 10.53%)

Test Case Distribution:

Time Interval	Percentage
0ms	~40%
1ms	~10%
2ms	~10%
3ms	~20%
4ms	~15%
5ms	~5%
6ms	~5%
7ms	~5%

Code Snippets:

```
#include <limits.h>
int min(int a, int b)
{
    return (a < b) ? a : b;
}
```

```
int newKeep = INT_MAX;
int newSwap = INT_MAX;
if (nums1[i] > nums2[i - 1] &&
    nums2[i] > nums1[i - 1])
{
    newKeep = min(newKeep, keep);
    newSwap = min(newSwap, swap + 1);
}
if (nums1[i] > nums2[i - 1] &&
    nums2[i] > nums1[i - 1])
{
    // ...
}
```

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>

#define INF 9999

void floydWarshall(int graph[10][10], int n)
{
    int i, j, k;

    // Updating shortest distances
    for(k = 0; k < n; k++)
    {
        for(i = 0; i < n; i++)
        {
            for(j = 0; j < n; j++)
            {
                if(graph[i][k] + graph[k][j] < graph[i][j])
                {
                    graph[i][j] = graph[i][k] + graph[k][j];
                }
            }
        }
    }

    printf("Shortest distance matrix:\n");

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
```

```

        {
            if(graph[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", graph[i][j]);
        }
        printf("\n");
    }
}

int main()
{
    int n, i, j;
    int graph[10][10];

    printf("Enter number of vertices:\n");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    printf("Use %d for no direct edge\n", INF);

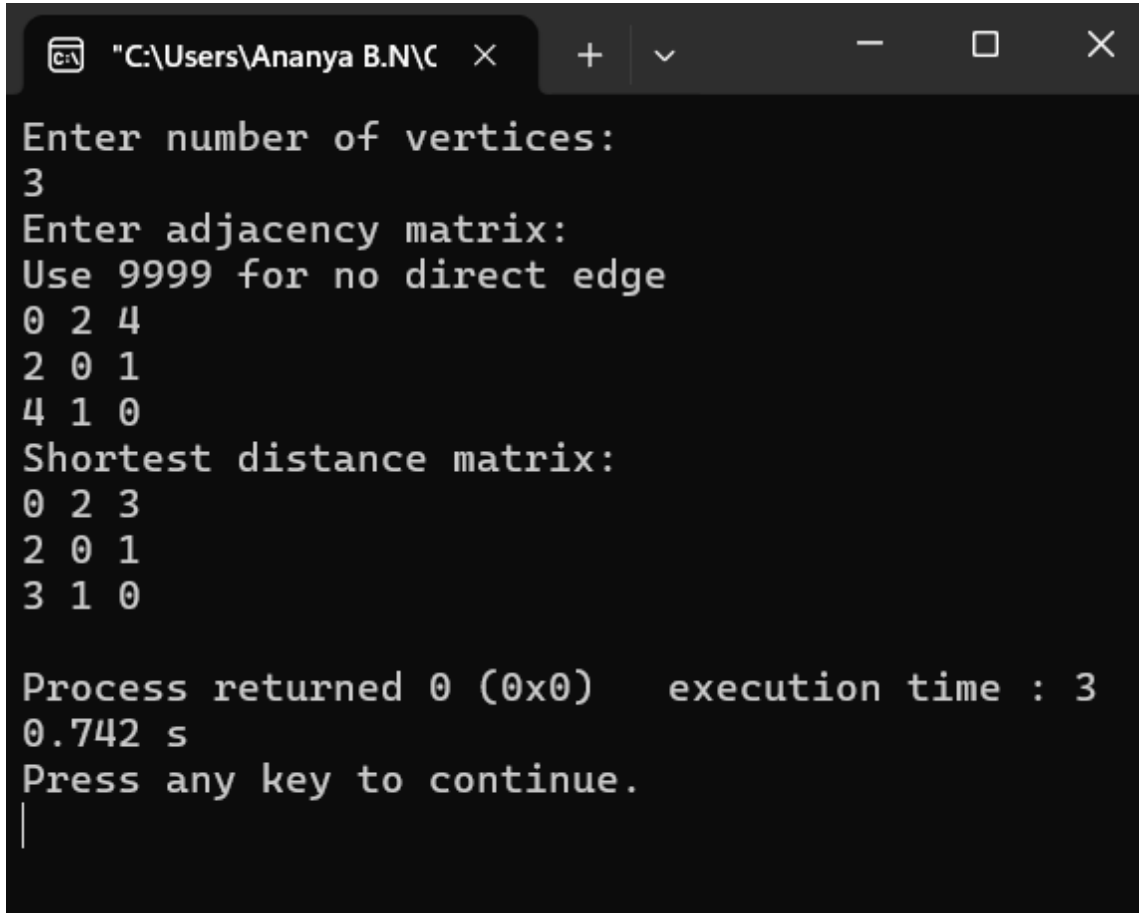
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    floydWarshall(graph, n);

```

```
    return 0;  
}
```

OUTPUT:



```
"C:\Users\Ananya B.N\C" × + ∨ − □ ×  
Enter number of vertices:  
3  
Enter adjacency matrix:  
Use 9999 for no direct edge  
0 2 4  
2 0 1  
4 1 0  
Shortest distance matrix:  
0 2 3  
2 0 1  
3 1 0  
  
Process returned 0 (0x0)    execution time : 3  
0.742 s  
Press any key to continue.  
|
```


Leetcode Problem (287.Duplicate Number):

```
int findDuplicate(int* nums, int numsSize)
{
    int slow = nums[0];
    int fast = nums[0];

    while (1)
    {
        slow = nums[slow];
        fast = nums[nums[fast]];

        if (slow == fast)
            break;
    }

    slow = nums[0];

    while (slow != fast)
    {
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}
```

OUTPUT:

```
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3
Input
nums =
[1,3,4,2,2]
Output
2
Expected
2
```

```
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3
Input
nums =
[3,1,3,4,2]
Output
3
Expected
3
```

The screenshot displays a coding platform interface with the following components:

- Problem List:** Shows the current problem and navigation options.
- Accepted:** Indicates that the solution has been accepted, with 59 / 59 testcases passed.
- Runtime and Memory:** Shows a runtime of 0 ms and memory usage of 14.98 MB (48.83% of the limit).
- Bar Chart:** A performance chart showing the distribution of runtime across different test cases.
- Code Editor:** Contains the C++ code for the `findDuplicate` function, which uses the slow-fast pointer technique to find the duplicate number.
- Testcase 1:** Shows the input `nums = [1,3,4,2,2]` and the output `2`.
- Footer:** Displays system information such as temperature (23°C), time (11:33 PM), and date (06-06-2025).

Lab program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100
#define INF 999999

void Prims(int n, int cost[MAX][MAX]) {
    int t[MAX][3];
    int mincost = 0;
    int near[MAX];

    int min = INF;
    int k = -1, l = -1;

    for (int r = 1; r <= n; r++) { //used to find the edge with the minimum cost
        for (int c = 1; c <= n; c++) {
            if (cost[r][c] < min && r != c) {
                min = cost[r][c];
                k = r;
                l = c;
            }
        }
    }

    t[1][1] = k;
    t[1][2] = l;
    mincost = cost[k][l];
```

```

for (int i = 1; i <= n; i++) {    //is the vertex closer to k or l
    if (cost[i][k] < cost[i][l]) {
        near[i] = k;
    } else {
        near[i] = l;
    }
}

```

```

near[k] = 0;
near[l] = 0;

```

```

for (int i = 2; i <= n - 1; i++) {
    int j = -1;
    min = INF;

```

```

    for (int m = 1; m <= n; m++) {
        if (near[m] != 0 && cost[m][near[m]] < min) {
            min = cost[m][near[m]];
            j = m;
        }
    }

```

```

t[i][1] = j;
t[i][2] = near[j];

```

```

mincost = mincost + cost[j][near[j]];

```

```

    near[j] = 0; // Mark vertex j as included in the tree

    for (int k_idx = 1; k_idx <= n; k_idx++) {
        if (near[k_idx] != 0 && cost[k_idx][near[k_idx]] > cost[k_idx][j]) {
            near[k_idx] = j;
        }
    }
}

printf("congrats\n");
printf("Edges in the Minimum Spanning Tree:\n");
for (int idx = 1; idx <= n - 1; idx++) {
    printf("%d -> %d\n", t[idx][1], t[idx][2]);
}
printf("mincost: %d\n", mincost);
}

int main() {
    int n, edges;
    int cost[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j)
                cost[i][j] = 0;
            else
                cost[i][j] = INF;
        }
    }
}

```

```

    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

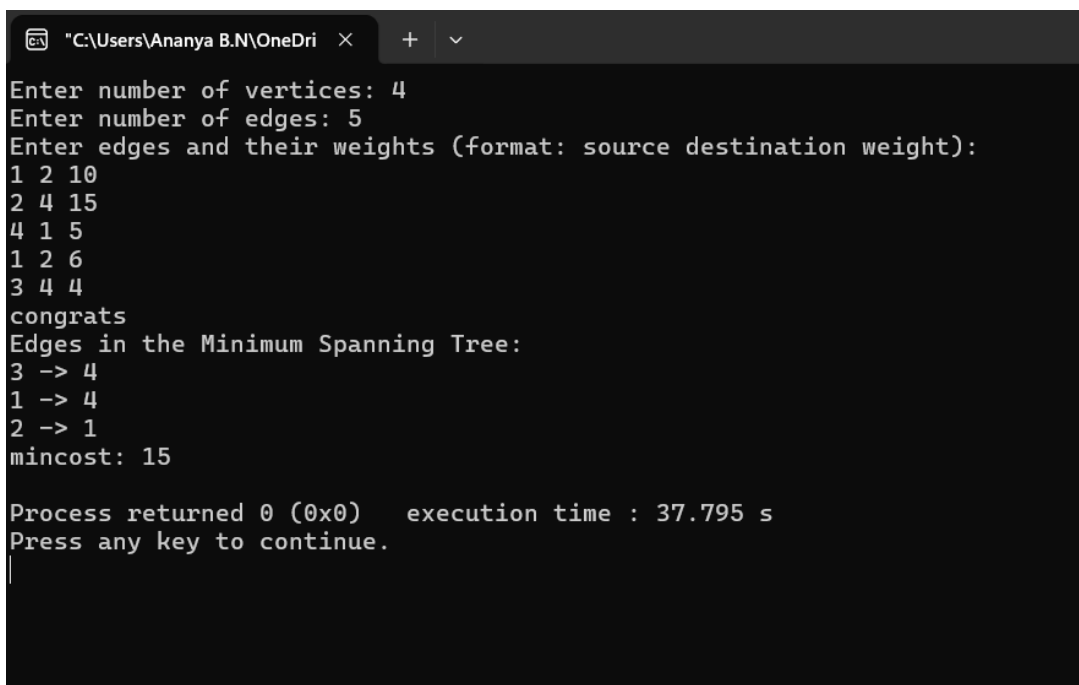
    printf("Enter edges and their weights (format: source destination weight):\n");
    for (int i = 0; i < edges; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        cost[u][v] = w;
        cost[v][u] = w; // Handled as an undirected graph
    }

    // Run the implementation
    Prims(n, cost);

    return 0;
}

```

OUTPUT:



```

C:\Users\Ananya B.N\OneDri x + v
Enter number of vertices: 4
Enter number of edges: 5
Enter edges and their weights (format: source destination weight):
1 2 10
2 4 15
4 1 5
1 2 6
3 4 4
congrats
Edges in the Minimum Spanning Tree:
3 -> 4
1 -> 4
2 -> 1
mincost: 15

Process returned 0 (0x0)   execution time : 37.795 s
Press any key to continue.
|

```

Lab program 8:

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

#define MAX 100

int parent[MAX];

int find(int i)
{
    while(parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int i, int j)
{
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}

int main()
{
    int n, cost[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
```

```

    {
        scanf("%d", &cost[i][j]);

        // Replace 0 with infinity (except diagonal)
        if(cost[i][j] == 0 && i != j)
            cost[i][j] = 999;
    }
}

```

```

for(int i = 0; i < n; i++)
    parent[i] = i;

```

```

int mincost = 0;
int edges = 0;

```

```

printf("\nEdges in MST:\n");

```

```

while(edges < n - 1)
{
    int min = 999, a = -1, b = -1;

    // Find minimum edge
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            if(cost[i][j] < min)
            {
                min = cost[i][j];
                a = i;
                b = j;
            }
        }
    }
}

```



```

        }
    }
}

int u = find(a);
int v = find(b);

if(u != v)
{
    printf("Edge %d: (%d - %d) cost = %d\n", edges + 1, a, b, min);
    mincost += min;
    unionSet(u, v);
    edges++;
}

cost[a][b] = cost[b][a] = 999;
}

printf("\nMinimum cost = %d\n", mincost);

return 0;
}

```

OUTPUT:

```
"C:\Users\Ananya B.N\OneDri  X  +  v
Enter number of vertices: 5
Enter number of edges: 7
Enter edges and their costs (format: source destination cost):
1 2 8
1 3 7
2 3 5
2 4 4
2 5 3
4 5 2
3 5 6
congrats
Edges in the Minimum Spanning Tree:
4 -> 5
2 -> 5
2 -> 3
1 -> 3
mincost: 17

Process returned 0 (0x0)   execution time : 57.419 s
Press any key to continue.
|
```

Lab program 9:

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>
void main()
{
    float p[100],w[100],x[100],m,total;
    int n,k;
    printf("enter the number of items");
    scanf("%d",&n);
    printf("enter the capacity of knapsack:");
    scanf("%f",&m);
    for(int i=0;i<n;i++)
    {
        printf("enter profit value %d: ",i+1);
        scanf("%f",&p[i]);
        x[i]=0;
        printf("enter weight value %d: ",i+1);
        scanf("%f",&w[i]);
    }
    int temp;
    for(int i=0;i<n-1;i++)
    {
        for(int j=i+1;j<n;j++)
        {
            if(p[i]/w[i]<p[j]/w[j])
            {
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
                temp=w[i];
                w[i]=w[j];
                w[j]=temp;
            }
        }
    }
}
```

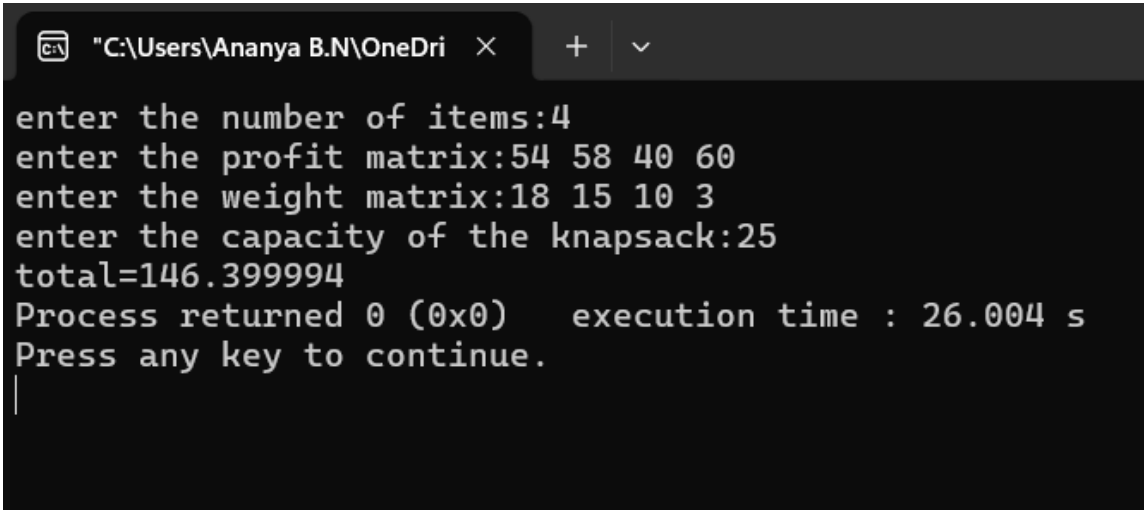
```

    }
    int rem_cap=m;
    for(k=0;k<n;k++)
    {
        if(w[k]>rem_cap)
        {
            break;
        }
        rem_cap=rem_cap-w[k];
        x[k]=1;
    }
    if(k<n)
    {
        x[k]=rem_cap/w[k];
    }
    for(int i=0;i<n;i++)
    {
        total=total+x[i]*p[i];
    }

    printf("total profit=%f",total);
}

```

OUTPUT:



```

C:\Users\Ananya B.N\OneDri >
enter the number of items:4
enter the profit matrix:54 58 40 60
enter the weight matrix:18 15 10 3
enter the capacity of the knapsack:25
total=146.399994
Process returned 0 (0x0)    execution time : 26.004 s
Press any key to continue.
|

```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>

void main()
{
    int c[100][100],s[100],dist[100],v,n;
    printf("enter the number of vertices:");
    scanf("%d",&n);
    printf("enter the cost between each vertex:");
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            scanf("%d",&c[i][j]);
        }
    }
    printf("enter the src vertex: ");
    scanf("%d",&v);
    for(int i=0;i<n;i++)
    {
        s[i]=0;
        dist[i]=c[v][i];
    }
    s[v]=1;
    dist[v]=0;
    for(int w=0;w<n-2;w++)
    {
        int min=dist[0];
        int u;
        for(u=0;u<n;u++)
        {
            if (dist[u]<min && s[u]==0)
            {
```

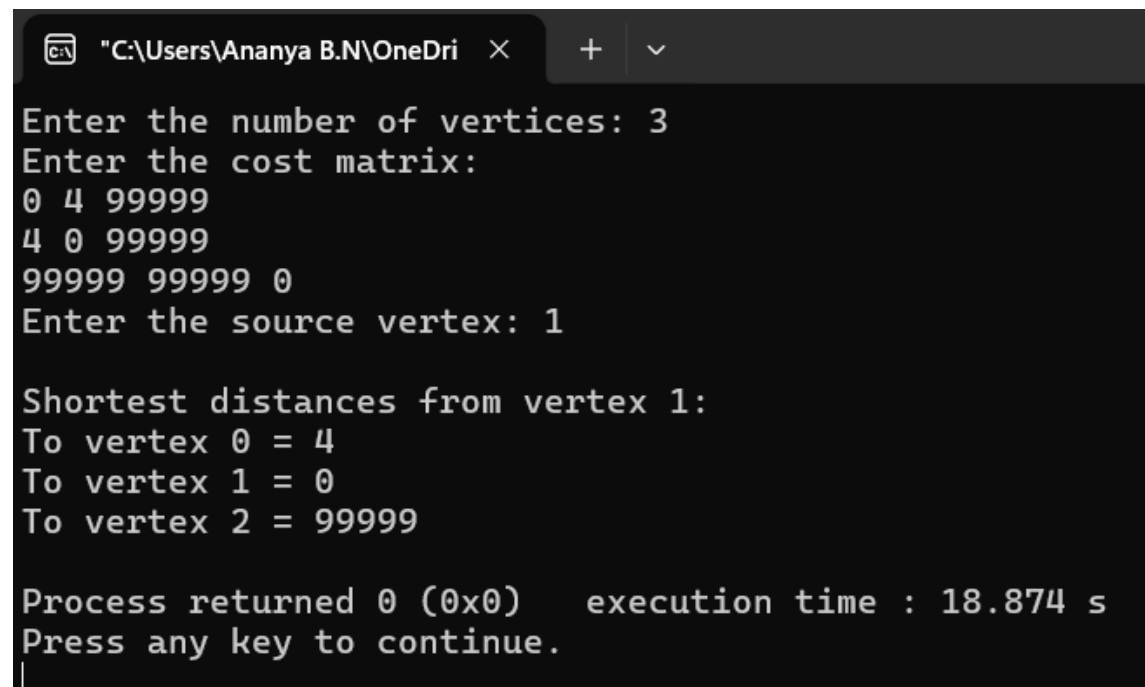
```

        min=u;
    }
}
s[min]=1;
for(int k=0;k<n;n++)
{
    if(dist[k]>(dist[k]+c[u][k]))
    {
        dist[k]=dist[k]+c[u][k];
    }
}

}
for(int i=0;i<n;i++)
{
    printf("dist[%d]=%d\n",i,dist[i]);
}
}

```

OUTPUT:



```

C:\Users\Ananya B.N\OneDri > Enter the number of vertices: 3
Enter the cost matrix:
0 4 99999
4 0 99999
99999 99999 0
Enter the source vertex: 1

Shortest distances from vertex 1:
To vertex 0 = 4
To vertex 1 = 0
To vertex 2 = 99999

Process returned 0 (0x0)   execution time : 18.874 s
Press any key to continue.

```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdbool.h>
bool place(int[], int);
void printSolution(int[], int);
void nQueens(int);
int main()
{
    int n;
    printf("Enter the number of queens: ");
    scanf("%d",&n);
    nQueens(n);
    return 0;
}

void nQueens(int n){
    int x[10];
    int count=0;
    int k=1;
    while(k!=0){
        x[k]=x[k]+1;
        while(x[k]<=n && !place(x,k)){
            x[k]=x[k]+1;
        }
        if(x[k]<=n){
            if(k==n){
                printSolution(x, n);
                printf("Solution found\n");
                count++;
            }else{
                k++;
            }
        }
    }
}
```

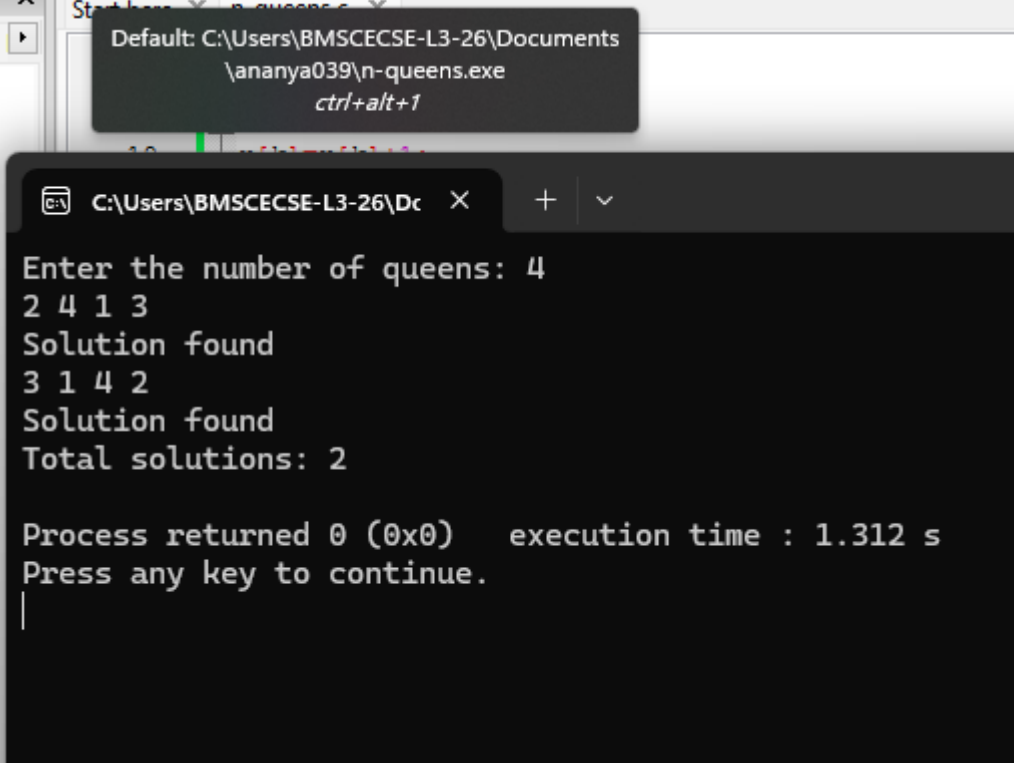
```

x[k]=0;
}
} else {
k--;
}
}
printf("Total solutions: %d\n", count);
}
bool place(int x[10], int k){
int i;
for(i=1;i<k;i++){
if((x[i]==x[k])||(i-x[i]==k-x[k])||(i+x[i]==k+x[k])){

return false;
}
}
return true;
}
void printSolution(int x[10], int n){
int i;
for(i=1;i<=n;i++){
printf("%d ", x[i]);
}
printf("\n");
}

```


OUTPUT:



The screenshot shows a Windows command prompt window with a dark background. The title bar indicates the file path: C:\Users\BMSCECSE-L3-26\Documents\ananya039\n-queens.exe. The command prompt shows the following text:

```
Enter the number of queens: 4
2 4 1 3
Solution found
3 1 4 2
Solution found
Total solutions: 2

Process returned 0 (0x0)   execution time : 1.312 s
Press any key to continue.
|
```